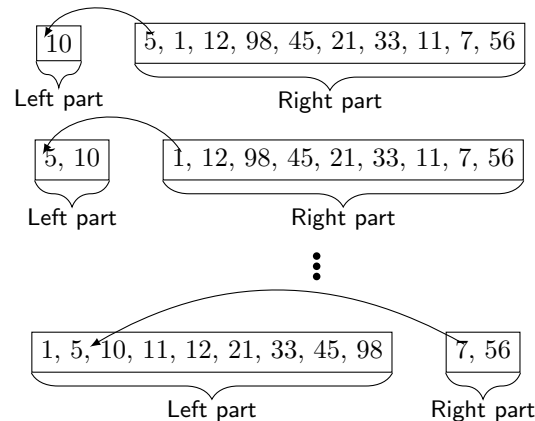


Insertion sort

Consider the process each player follows when arranging the cards dealt in a game, where cards are distributed one at a time among players. Each player selects a card and sorts it by inserting the new card into its proper position among the cards they already hold. The cards in hand are always kept in sorted order. When an incoming card is received, it is compared with each card in hand until it encounters a card of higher value, let's call it x . At that point, all cards from position x onward shift one place to the right, allowing the incoming card to be inserted in the position previously occupied by card x .

To develop an insertion sorting algorithm, we can think of the array of numbers as divided into two subsets: one that is sorted and the other that remains unsorted. Initially, the left partition of the array consists of a single element, while the right partition contains all the remaining elements. We will simulate a card sorting process to gradually insert the elements from the right partition into their appropriate positions within the sorted left partition, starting with the leftmost element of the right partition. At each step, the size of the left partition increases while the right partition decreases. Once the right partition is empty, the array will be fully sorted.

Let us illustrate it with an example. Consider the array given below;



The correctness proof of insertion sort depends on preserving the sorted order of the left partition of the array and that the cardinality of this part grows from one to n . Initially, only one element is in the left partition of the array. Therefore, it preserves the sortedness of the partition. Assume that after k iterations, the left partition still preserves its sortedness characteristic. Now we consider $k + 1$ iteration. Assume that we start with the first element of the left partition. Each

element is compared with the incoming element and remains in the position until we encounter the element x , which is greater than the incoming element. Then, we shift all remaining elements, including x , one position to the right. It creates a hole or a gap at the previous position of x in the left partition. We then drop the incoming element into the gap. The process does not disturb the relative order of the existing elements of the left partition. However, the incoming element drops into the previous position of x . Since the incoming element is smaller than x , it is smaller than all elements to the right of x in the left partition. Therefore, after dropping the incoming element, the characteristics of the left part remain unchanged. Notice that the argument works correctly whether we compare starting from the leftmost or the rightmost aspect of the left partition. The proof of correctness for insertion sort relies on maintaining the sorted order of the left partition of the array, which grows from one element to n . Initially, the left partition contains only one element, ensuring its sortedness.

Assume that after k iterations, the left partition still preserves its sorted characteristics. Now, let's consider the $k + 1$ iteration. We start with the first element of the left partition. Each element is compared with the incoming element, and it remains in its position until we encounter an element x that is greater than the incoming element.

At this point, we shift all elements, including x , one position to the right, creating a gap where x was located in the left partition. We then place the incoming element into this gap. This process does not disturb the relative order of the existing elements in the left partition. Since the incoming element is smaller than x , it must also be smaller than all the elements to the right of x in the left partition. Consequently, after placing the incoming element, the sorted characteristics of the left partition is preserved.

Note that this argument holds true regardless of whether we start comparing from the leftmost or the rightmost end of the left partition.

We begin by moving every element in the left subarray to the right after identifying the relative position of the new incoming element from the right subarray, shifting it one position to the right. In the worst-case scenario, each iteration may necessitate that every element be shifted one position to the right, with this potentially occurring for the entire length of the subarray. If the subarray contains i elements, the total number of shifts amounts to i . Consequently, the worst-case running time can be expressed as the summation of i for $1 \leq i < n$, resulting in a complexity of $O(n^2)$.